

工學 學士 學位論文

Optimization for Rowhammer Mitigation utilizing Internal DRAM Topologies

(DRAM 내부 정보를 활용한 Rowhammer
방어 전략 최적화 제안)

2023년 2월

서울대학교 공과 대학

電氣·情報工學部

이준기

Optimiziation for Rowhammer Mitigation utilizing Internal DRAM Topologies

指導教授 심재웅

이 논문을 공학 학사학위 논문으로 제출함.

서울대학교 공과 대학

電氣·情報工學部

이준기

이준기의 공학 학사학위 논문을 인준함.

2022년 12월 22일

지도교수 심재웅 (인)

ABSTRACT

RowHammer is a serious security threat to computing systems using DRAM since the first experimental analysis in 2014 [1]. Through repetitive activation of the DRAM row, electromagnetic inference causes charge leakage in DRAM cells. Even worse, technology scaling makes DRAM even more vulnerable to RowHammer attack [2]. Many RowHammer mitigation systems have been proposed, but a lack of understanding of the internal DRAM structure poses inefficiency in the system. In this paper, I point out some optimization points to enhance the current state-of-the-art RowHammer mitigation schemes using internal DRAM topologies in DDR4 memory system: Graphene [3] and Hydra [4]. I evaluate speedup between baseline and optimized version in Macsim with Ramulator [5], [6] (cycle-accurate and trace-driven simulator, with 2-channel DDR4 configuration) with 2 synthetic adversarial attack patterns and 3 memory-intensive workloads from SPEC CPU 2017 under the RowHammer threshold of 1K. Results show that with 4% additional CAM in Graphene, an additional refresh was reduced by nearly 50% in adversarial patterns and 64.7% in normal workloads with a performance overhead reduced by 65.7%. In the case of Hydra, an additional refresh was reduced by 91.4% in adversarial pattern and 98.1% in normal workloads with average performance overhead reduced by 71.7%.

Keyword: DRAM, DDR4, Reliability, RowHammer, Graphene, Hydra

Contents

1. Introduction	1
2. Background	3
2.1 DRAM Overview	3
2.1.1 DRAM Organization	3
2.1.2 Timing	4
2.2 RowHammer	5
2.2.1 Phenomenon	5
2.2.2 Existing Solutions and Limitations	5
3. Optimization Design	8
3.1 Graphene	8
3.1.1 Algorithm & Modification	8
3.1.2 Security Analysis of Modified Algorithm	10
3.2 Hydra	11
3.2.1 Algorithm & Modification	11
4. Evaluation	12
4.1 Experimental Setup	12
4.1.1 Simulator Setup	12
4.1.2 Workload Setup	12
4.2 Evaluation Result	13
4.2.1 Storage Overhead	13
4.2.2 Overall Performance	14
5. Related Work	16

6. Conclusion and Future Work	18
Reference	19

List of Figures

2.1	Overview of DDR memory system	3
2.2	DRAM chip organization	4
2.3	Graphene Overview	6
2.4	Hydra Overview	7
4.1	The number of refreshes as mitigation in two synthetic workloads	14
4.2	IPC normalized to baseline (no mitigation) in SPEC2017 memory-intensive workloads	14
4.3	Overhead analysis in SPEC2017 workloads	15

List of Tables

2.1	Typical DDR4 timing parameters used in paper	5
4.1	Simulation setting of Macsim and Ramulator	13
4.2	Storage Overhead	13

Chapter 1

Introduction

DRAM has long been a popular choice for main memory in a computing system, and its cells are becoming smaller and closer to each other due to technology scaling. This makes DRAM vulnerable to inter-cell interference. *RowHammer* (RH) attack is one type of such interference, where a burst of ACT command to a single DRAM row causes a charge leak in adjacent rows.

This problem is a major threat to DRAM reliability, and thus many solutions have been proposed by both industry and academia. Recent protection schemes are based on architectural solutions, which implement hardware logic to detect hammering rows and handle them proactively. This class of mitigation scheme can be divided into two parts: probabilistic [1] and counter-based [3], [4], [7], [8]. One of the most famous probabilistic schemes is PARA [1], which refreshes adjacent rows with small predefined probability. Although it doesn't require complex hardware logic, it invokes a substantial amount of unnecessary refreshes to ensure the required security level. However, counter-based protection only refreshes victim rows deterministically, making it an attractive choice. *Graphene* [3] is best known counter-based protection scheme which uses Misra-Gries streaming algorithm [9] to track hammering rows. It accurately tracks a set of rows that were activated more than a certain threshold from incoming commands.

The metric used to characterize how a DRAM chip is vulnerable to RH attack is $Flip_{th}$, which indicates the minimum number of row activations (ACT) required to induce a bit-flip in adjacent rows. Recent studies show that $Flip_{th}$ has dropped from 139K (DDR3) to 50K in DDR4 and 4.8K for LPDDR4 [1], [10], [2]. As technology continues to scale, $Flip_{th}$ will likely decrease further, even lower than 1K. *Hydra* [4] is a counter-based mitigation scheme targeting such ultra-low threshold with acceptable storage overhead. It uses the SRAM counter to track ACTs of the DRAM row group and uses the DRAM counter to track each row when the group

CHAPTER 1. INTRODUCTION

counter reaches a certain threshold.

Although promising, all of these mitigation strategies are inefficient in various aspects because there are no public documents available for internal DRAM microarchitecture. In this paper, I point out some optimization ways of *Graphene* and *Hydra* if such internal DRAM topologies are available. I implemented *Graphene*, *Hydra*, and optimized version of each in Macsim [5] and Ramulator [6]. Using these simulators, I evaluated storage overhead, the number of additional refreshes, and performance in synthetic and normal workloads such as SPEC CPU 2017. Below are the key contributions of this work.

- *Graphene* optimization: As DRAM’s internal row mapping is not known, the hardware tracker doesn’t know the adjacent rows of a hammered row. So tracking algorithm of *Graphene* always has to assume the attack is done as *double-sided*, where one victim row is adjacent to two hammered rows. I tracked victim rows instead of hammering rows and analyze the benefits and security of the modified algorithm.
- *Hydra* optimization: Recent comprehensive analysis of DDR4 device showed that due to design and manufacturing factors, $Flip_{th}$ variation exists across rows in DRAM. In the optimized version of *Hydra*, I stored the encoded $Flip_{th}$ value in each row’s counter in DRAM.
- Evaluation: I implemented the above optimization points in a cycle-level simulator and evaluated storage overhead, additional refresh, and performance. In *Graphene*, an additional refresh was reduced by 50% in attack patterns and average performance overhead was reduced by 65.7% in normal workloads with a negligible increase in table size. In the case of *Hydra*, an additional refresh was reduced by 91.4% in attack patterns and performance overhead was reduced by 71.7% in normal workloads.

In this paper, I first illustrate the background of DRAM structure and RH mitigation schemes of *Graphene* and *Hydra* in Chapter 2. In Chapter 3, I explain improvement points and implementation details with security analysis. After that, I compare storage overhead, the number of an additional refreshes, and the performance (IPC) of each scheme’s baseline and optimized version in Chapter 4. Finally, Chapter 5 summarizes related work in progress, and Chapter 6 concludes this paper and suggests possible future works.

Chapter 2

Background

2.1 DRAM Overview

2.1.1 DRAM Organization

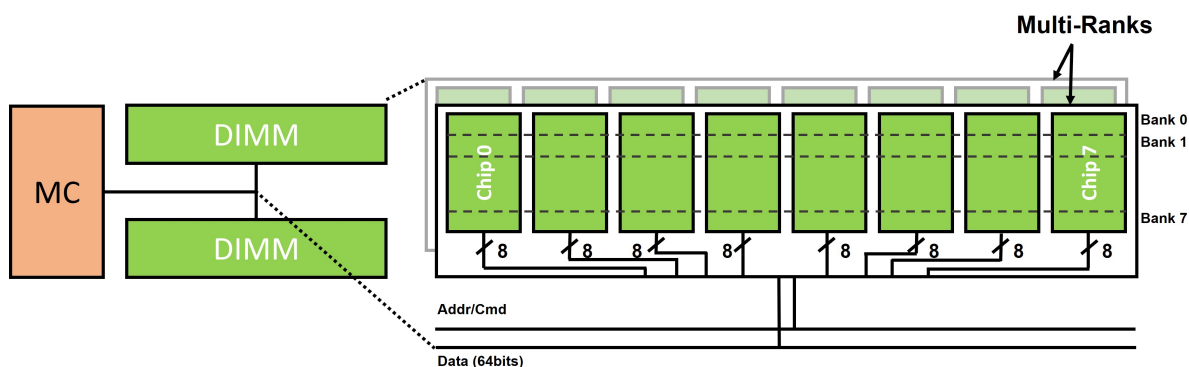


Figure 2.1: Overview of DDR memory system

Typical DDR memory systems consist of multiple memory controllers (MCs). Single MC controls channel(s) and each channel consist of single or multiple ranks (See Figure 2.1), where several DRAM devices (chips) operate in lock-step. Each DRAM chip accepts the address with the command and outputs data in the DQ pin. Several independent banks reside in each chip, and adjacent data is interleaved into different banks in order to hide memory access latency.

Figure 2.2 shows the hierarchy inside a DRAM chip. Chip has several banks, and each bank consists of multiple MATs. MAT is a matrix of cells (typically 512×512 or 1024×1024) each connected by wordline (WL) and bitline (BL). MATs sharing local sense amplifiers form subarray. When a row address comes to the global row decoder, it selects the subarray row de-

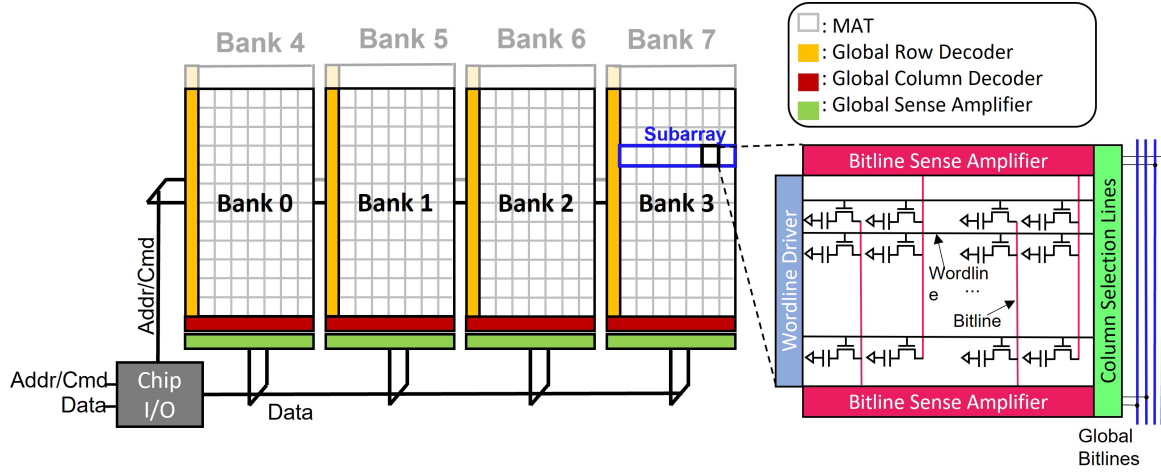


Figure 2.2: DRAM chip organization

coder which in turn activates WL (i.e. target address) via wordline drivers. When local WL is activated, data in each cell is read from BL to a local sense amplifier. Since each sense amplifier is much larger than each cell, putting sense amplifiers in one place can make a composition of cells inside MATs sparse. Since sense amplifier is symmetric, it can be connected at top of MAT as well as the bottom. This architecture is called *open bitline architecture* and can lead to a more compact design of MATs. After data is read to a local sense amplifier, a column select line chooses data corresponding to the column address and the data is sent to the global sense amplifier by global bitlines.

2.1.2 Timing

Due to the master-slave interface between MC and DRAM device, MC has to be stalled for a predefined amount of time between each command. Also, the capacitor in the DRAM cell slowly leaks when idle. The amount of time a DRAM cell can retain its data is called *retention time*. MC prevents cells from losing their data by periodically sending a restore command which is called *refresh*. DDR4 standard [11] specifies timing parameters related to the refresh command, which is the refresh interval. Refresh interval (**tREFI**) is a period of refresh command sent to DRAM to refresh DRAM row (or group of rows). In order to preserve data, the refresh period for a given row (**tREFW**) must be shorter than the minimum retention time for entire cells in DRAM. This parameter is vendor-specific, and the JEDEC standard doesn't specify tREFW. Typical values of tREFW, tREFI, and other timing parameters for DRAM read/write are summarized in table 2.1.

Table 2.1: Typical DDR4 timing parameters used in paper

Term	Definition	Value
tREFI	Refresh period	7.8 μ s
tREFW	Refresh period for single row	64ms
tRFC	Refresh command duration	350ns
tRC	Row cycle time	45ns

2.2 RowHammer

2.2.1 Phenomenon

Repetitive activation of a single row makes a small number of electrons flow at adjacent cells, making its cells discharge faster. This causes bits to be flipped, enabling an attacker to manipulate data without accessing them. Rows repetitively accessed to cause bitflip is called *aggressor rows*, and rows affected by aggressor rows are called *victim rows*. A minimum number of ACT commands required to induce bitflip is called $Flip_{th}$. Since $Flip_{th}$ is getting lower due to technology scaling in DRAM, finding an efficient solution is becoming more important.

2.2.2 Existing Solutions and Limitations

Numerous solutions have been proposed by both academia and industry. Recent researches focus on architectural scheme, which implements hardware logic inside MC or DRAM (or both) to detect aggressor rows at run time and handle them. PARA [1], a probabilistic approach to refresh adjacent rows with small predefined probability, doesn't require complex tracking logic but suffers from a significant amount of unnecessary refresh which degrades performance seriously. Counter-based approach, on the other hand, reserve counter for aggressor rows and refreshes victim rows when the counter reaches a predefined threshold. TWiCe [7] proposed *Near Row Refresh* (NRR) command for handling such victim refresh: When MC sends NRR, the DRAM device finds and refreshes two adjacent rows of the command. There are two inefficiencies in this approach. First, MC doesn't know which physical row corresponds to a given logical row, so the tracker must assume the most malicious kinds of attack: a single victim row between two aggressor rows (double-sided). Thus, the bitflip threshold is halved. Second, when the bitflip threshold is reached, MC just sends the NRR command without feedback from the DRAM chip. So MC doesn't know which rows were refreshed, enabling attackers to utilize mitigation action itself for an attack: Half-double [12]. Below I present two state-of-the-art counter-based approaches and their limitations.

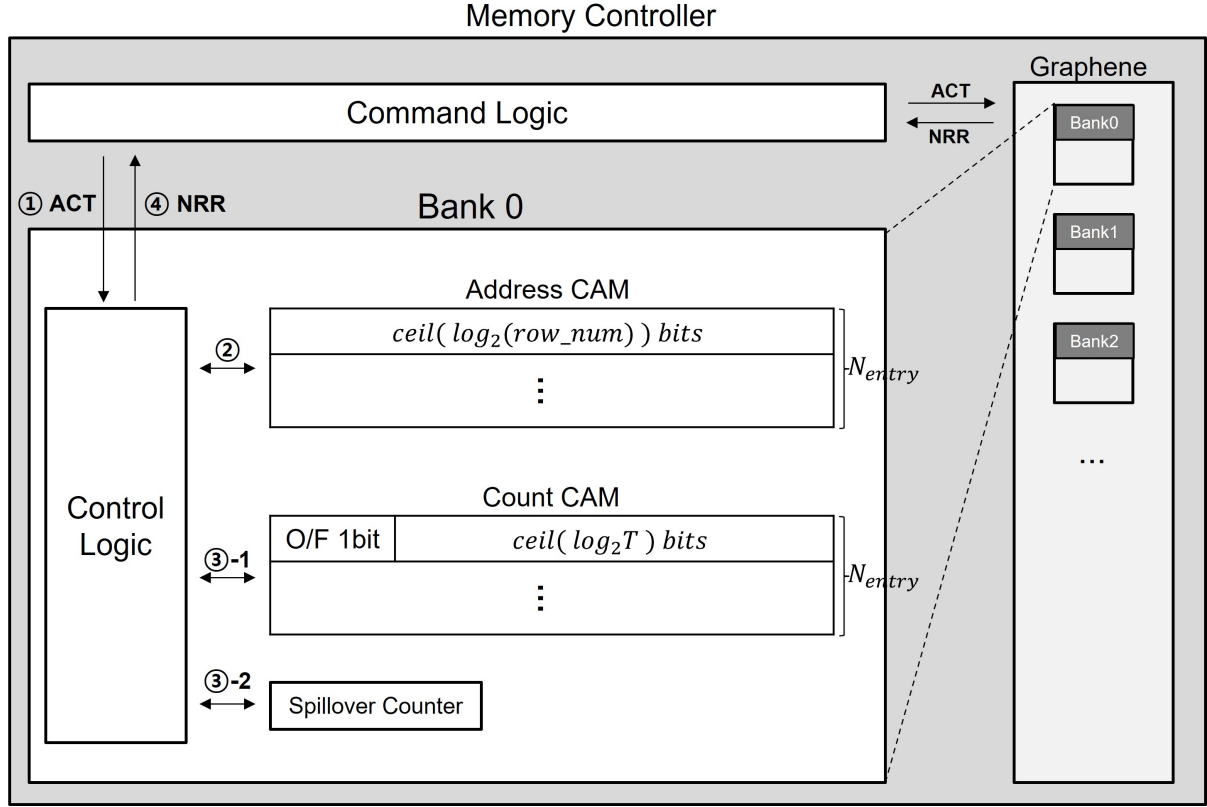


Figure 2.3: Graphene Overview

Graphene utilizes the Misra-Gries algorithm [9] to track the number of ACTs each row received during tREFW. Figure 2.3 shows an overview of Graphene. It maintains counter CAM for incoming ACTs in MC per each bank. ① ACTs goes into corresponding bank, and ② consults its address CAM. In case of hit (③ -1), it accesses count CAM and increments the counter by 1. If the entry is not in count CAM, it then checks if any entry in count CAM has the same value as the spillover counter (③-2). If an entry is found, it evicts it and increments a counter. If not found, it increments the spillover counter. If the incremented counter matches the predefined threshold T (④-2), Graphene control logic sends out an NRR command. Although simple and intuitive, MC must assume that every attack is double-sided and T is half of $Flip_{th}$. Due to the steady trend of decreasing $Flip_{th}$, T is also decreased, which makes N_{entry} bigger. This leads to a bigger, slower Count CAM.

Hydra is a counter-based mitigation scheme targeting low $Flip_{th}$ with small storage overhead. It uses a hybrid scheme of two counter tables each with a predefined threshold: SRAM Group-Count Table (GCT) with T_G and DRAM Row-Count Table with T_H . GCT is an SRAM counter indexed by row address which is used to filter non-malicious attacks. All rows are statically mapped to a single entry in GCT, forming a *Row-Group*. RCT is a predefined region inside DRAM that saves counters for each row. Counters are untagged and can record values up

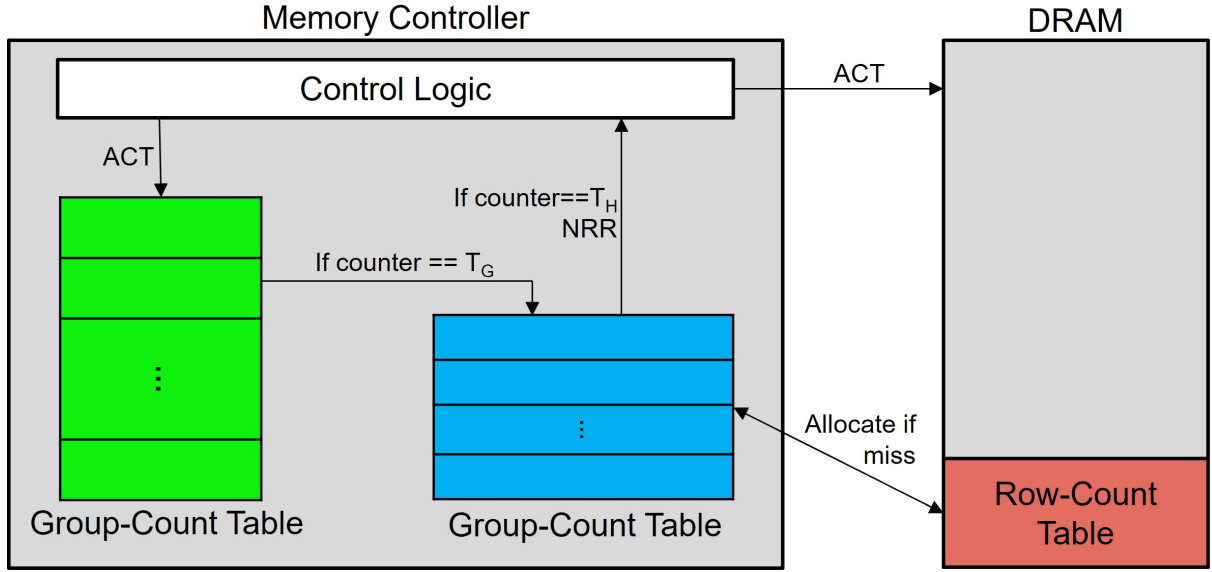


Figure 2.4: Hydra Overview

to T_H . Figure 2.4 shows a high-level overview of Hydra. At run time, each GCT entry stores aggregated activation count of corresponding row-group. When GCT entry reaches T_G , Hydra switches from group counting to per-row counting for rows in the group. All counters for row-group are initialized as T_G , and all subsequent access of rows in the group goes into RCT instead of GCT. When the RCT counter reaches T_H , Hydra control logic sends out an NRR command. To reduce the performance overhead of extra DRAM access when updating the RCT counter, Hydra used Row-Count Cache (RCC) to cache each frequently accessed RCT entry. Although this scheme scales well with decreasing $Flip_{th}$, it uses the same NRR-based mitigation as Graphene which leads to the halved threshold and possible security breakage.

Chapter 3

Optimization Design

The underlying source of inefficiency in current state-of-the-art RH mitigation schemes is that MC doesn't know the mapping between physical to logical rows. Due to rapid technology scaling in DRAM foundry, current DRAM devices may have faulty cells due to manufacturing imperfection. DRAM vendors correct such errors by replacing rows containing faulty cells with fault-free rows. This approach is often called row sparing and has been widely adopted by vendors. Each DRAM bank has spare rows, and when faulty cells are found at the test phase after the manufacturing process, test program *remaps* faulty row's address to fault-free spare row and saves remapping information into programmable memory (remapping table). At runtime, the global row decoder consults the remapping table and finds the corresponding row. Also, the majority of DRAM devices use a simple hash function between logical row address (sent by MC) and physical row address (actual row layout) and this hash function differ between vendors and even products within the same vendor. All the above schemes complicate MC to find out actual adjacent rows of aggressor row, making the RH mitigation scheme inefficient. In this paper, I redesigned the state-of-the-art mitigation systems assuming MC knows the mapping between physical to logical rows.

3.1 Graphene

3.1.1 Algorithm & Modification

As explained in 2.2.2 Graphene uses CAM counter (per bank) to track activation count for row addresses during the last $\mathbf{W}$ ACTs. When the counter reaches $\mathbf{T}$, Graphene control logic

CHAPTER 3. OPTIMIZATION DESIGN

signals mitigator to send NRR for aggressor row. In order to implement the above scheme, the CAM counter must track N_{entry} entries that follows inequality 3.1 [3]:

$$N_{entry} > \frac{W}{T} - 1 \quad (3.1)$$

There are two things to consider when setting T at Graphene. First, Graphene refreshes its CAM counter (counter refresh) at every t_{REFW} . This counter refresh is asynchronous to individual rows, so a maximum $2(T - 1)$ number of ACTs can go through Graphene without invoking mitigation logic ($T-1$ ACTs between row refresh and counter refresh, another $T-1$ ACTs between counter refresh and next row refresh). Also, since every ACTs is considered as part of a double-hammering attack pattern, Graphene must use a threshold of $Flip_{th}/2$. Considering both points in the above discussion, we get the following inequality 3.2.

$$2(T - 1) < \frac{Flip_{th}}{2} \implies T < \frac{Flip_{th}}{4} + 1 \quad (3.2)$$

If MC is extended to know logical to physical row mapping, we can track victim rows instead of aggressor rows. The optimized version of Graphene (*Graphene-op*) uses the same algorithm, but instead of focusing on the aggressor, it determines the victims of ACT command, updates counter, and sends *refresh* command instead of NRR. We don't have to consider every ACTs as part of the double-hammering pattern, instead of using a halved threshold. However, since we always have to count two victims for a single ACT, the window size is doubled. So we get inequality 3.3 for Graphene-op.

$$\begin{aligned} 2(T - 1) < Flip_{th} &\implies T < \frac{Flip_{th}}{2} + 1 \\ N_{entry} &> \frac{2W}{T} - 1 \end{aligned} \quad (3.3)$$

Note that although the number of tracking elements is doubled, T is also doubled, making Graphene-op's CAM counter entry (N_{entry}) the same as Graphene. In addition, a Half-double attack is impossible in Graphene-op since it doesn't use NRR command. In conclusion, Graphene-op uses a threshold twice the size of Graphene without incurring almost no storage overhead and effectively blocks Half-double attacks.

3.1.2 Security Analysis of Modified Algorithm

Here I show that Graphene-op successfully mitigates RH by following the same flow as Graphene.

I start by proving the following theorem:

Theorem. *Tracking victim rows of last W aggressive ACTs in size of N_{entry} table and refreshing entry whenever it becomes multiple of T prevents any row's actual count from reaching T without incurring refresh*¹

Lemma 1. The (estimated) count of every entry in the table is equal to or greater than the actual count of any victim row.

Lemma 2. Spillover count cannot exceed $\frac{2W}{N_{entry}+1}$

Proof of Lemma 1. By algorithm introduced in section 2.2.2 and modification in section 3.1.1, the only way an estimated count can become smaller than an actual count is when it is evicted. When it is evicted, its estimated count must be equal to the spillover count. Between eviction and reentering, it will update the spillover counter. When it is written back, it will set its counter to spillover counter+1. $\square$

Proof of Lemma 2. Since every arrived ACTs increments the sum of the spillover counter and counters in the table by 2, the sum of the spillover counter and counters in the table has a maximum of $2W$. Since the spillover counter should be smaller or equal to a minimum value in the counters in the table, it will have a maximum value when all entries in the table have the same value as the spillover counter: $\frac{2W}{N_{entry}+1}$ $\square$

Proof of Theorem. When the actual count of victim row X turns $T-1$ to T , row X 's estimated count should be T or higher due to Lemma 1. So, refresh action must be performed or already has been performed in the past. By inequality 3.3 and Lemma 2, the spillover counter is always smaller than T and thus the entry is never evicted from that moment. So, when another T number of ACTs affects row X , the estimated counter would have increased by exactly T amount, and refresh action is performed accordingly. $\square$

¹actual count for a given row refers to the number of ACTs sent to any adjacent aggressor rows between refresh command

3.2 Hydra

3.2.1 Algorithm & Modification

Recent studies [13] showed that due to manufacturing factors and design factors in DRAM foundries, $Flip_{th}$ of each row differs greatly. In an optimized version of Hydra (Hydra-op), I encoded the unique $Flip_{th}$ of each row assuming it can be known at the test phase of the DRAM manufacturing process and embedded those metadata along with the row’s RCT counter. Since RCT size is less than 1% of typical DRAM, this method incurs a small storage overhead. There is also little performance degradation, as each row’s counter and metadata can be placed in the same cache line. Although tracking victim rows is orthogonal to adding threshold metadata for each row in the DRAM counter, I did not implement it in the current paper due to its obvious speedup. However, current work can easily be extended to incorporate both optimizations.

Chapter 4

Evaluation

This chapter deals with workloads and configuration I used for simulation, and compares storage overhead and performance result between the normal and optimized version of Graphene and Hydra.

4.1 Experimental Setup

4.1.1 Simulator Setup

I implemented Graphene, Hydra, and optimized versions in Macsim [5] with Ramulator [6], a cycle-accurate trace-driven simulator. I consider NRR as successive activation of two rows and modified Ramulator to support NRR command with timing parameters set accordingly. I configured Ramulator to follow the organization of the Micron DDR4 8Gb x8 chip and set ACT, PRE, RD, and WR timing accurately. Table 4.1 shows the detailed experimental configuration for Ramulator and Macsim.

4.1.2 Workload Setup

I evaluate baseline Graphene, Hydra, and each optimized version using 2 synthetic attack patterns and 3 normal workloads from SPEC2017 rate benchmark[14]. The three most memory-intensive benchmarks were chosen according to workload characterization in Hydra: parest, blender, and cactuBSSN. These benchmarks showed the highest number of unique rows which received more than 250 ACTs during tREFW. To compare the performance of different de-

Table 4.1: Simulation setting of Macsim and Ramulator

MACSIM CPU Configuration	
Core	4-core 3.2GHz
Private L1 Cache (set-way)	Icache 4KB (8-8), Dcache 16KB (32-8)
Private L2 Cache (set-way)	128KB (128-16)
Shared LLC (set-way)	1MB (1024-16)
Trace maximum	80M
Ramulator Memory Configuration	
Speed	DDR4-2400R
Memory bus speed	12GHz (2.4GHz DDR)
Organization	DDR4 8Gb x8
Configuration	2ch-1ra-4bg-4ba
Capacity	16Gb
Scheduling	FCFS
Row Policy	Opened row
$Flip_{th}$	1K
Hydra-op $Flip_{th}$ distribution	Uniform [1K, 21K]

signs under RH attack, two synthetic attack patterns were used: Repetitively accessing two rows apart far from each other and double-hammering attack.

4.2 Evaluation Result

4.2.1 Storage Overhead

Table 4.2 shows the storage overhead of 4 designs. Increased CAM size in Graphene-op is because it uses doubled threshold compared to Graphene and thus bit-width of the counter is increased accordingly. For Hydra-op, I assumed each row's $Flip_{th}$ is a random variable with a distribution of uniform [1K, 21K] and saved each value alongside the counter in RCT using the same bit-width. Thus, DRAM storage is doubled but still

Table 4.2: Storage Overhead

Design	Storage size (Byte)	Type
Graphene	533,400	CAM
Graphene-op	554,736	CAM
Hydra	56,576	SRAM
	2,097,152 (0.1%)	DRAM
Hydra-op	66,560	SRAM
	4,194,304 (0.2%)	DRAM

negligible (0.2%) compared to the total size of DRAM used in the simulation. In addition, each RCC entry is extended to save each row's threshold metadata, leading to a 17% increase in total size.

CHAPTER 4. EVALUATION

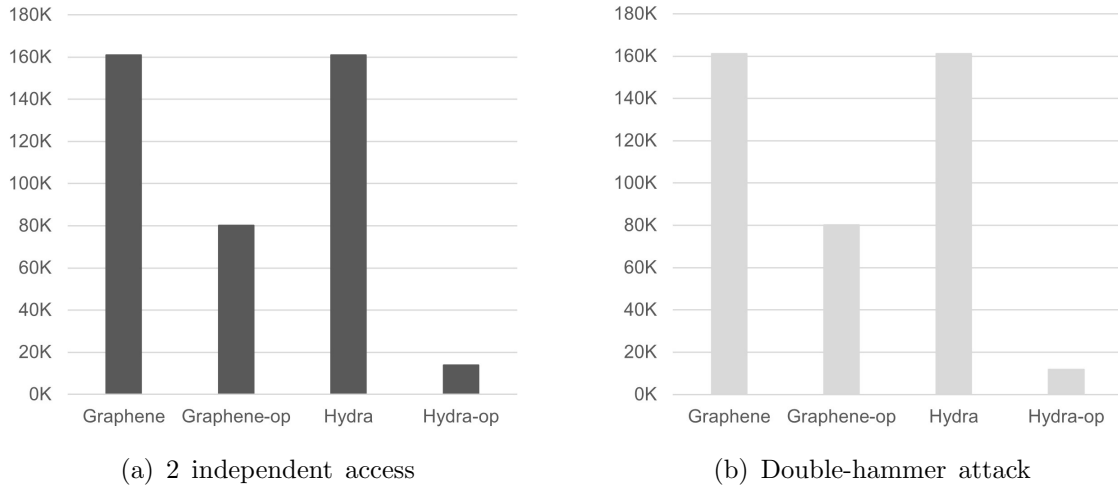


Figure 4.1: The number of refreshes as mitigation in two synthetic workloads

4.2.2 Overall Performance

To comprehensively evaluate the performance of optimized versus non-optimized versions, I first evaluate the number of refresh actions triggered by RH mitigation schemes in malicious workloads. Figure 4.1 shows the number of RH refreshes in each of four designs for 2 independent row hammering (4.1a) and double-hammering (4.1b) ¹. Regardless of the attack pattern, the number of refreshes in Graphene-op is reduced by 50%. This is because Graphene-op uses twice the larger threshold than Graphene. In the case of Hydra-op, the number of refreshes is reduced by 91.4%. This is because the number of refreshes is inversely proportional to the predefined threshold and the average threshold of Hydra-op is $11\times$ of non-optimized Hydra.

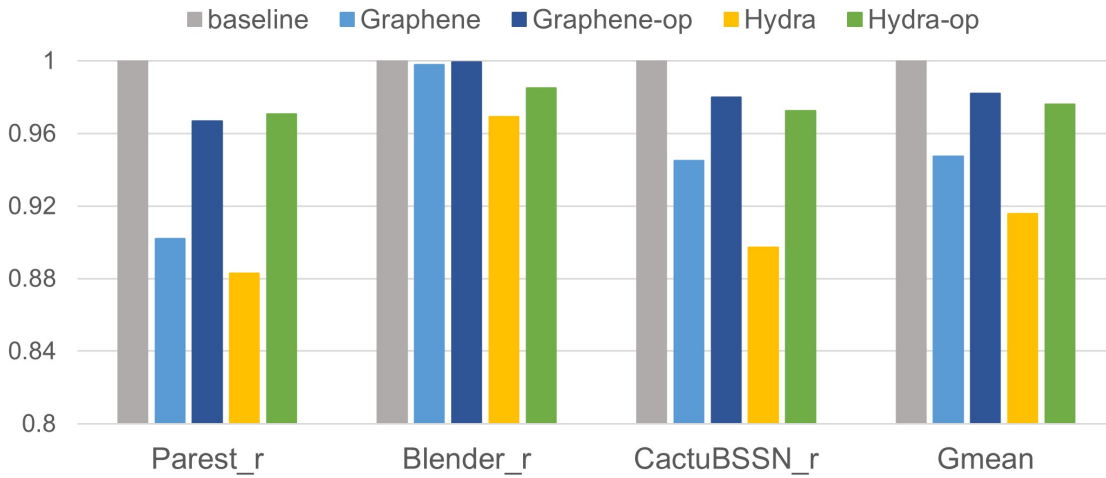


Figure 4.2: IPC normalized to baseline (no mitigation) in SPEC2017 memory-intensive workloads

¹Due to random distribution of each row's $Flip_{th}$, simulation of Hydra-op is repeated 100 times and used average

CHAPTER 4. EVALUATION

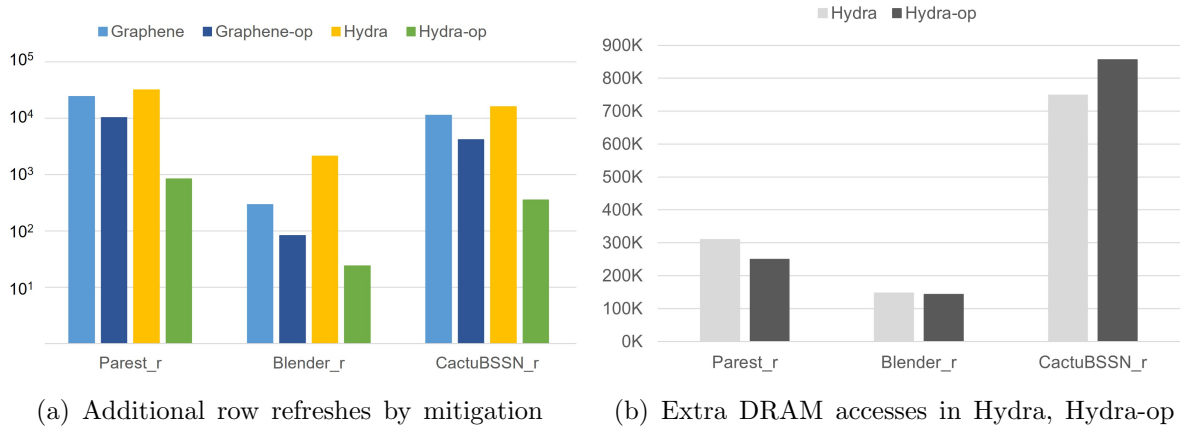


Figure 4.3: Overhead analysis in SPEC2017 workloads

In the case of benign accesses, I evaluate the IPC of memory-intensive workloads explained in 4.1.2 for 4 designs. Figure 4.2 shows IPC normalized to baseline (no RH mitigation scheme implemented). Both optimized versions of Graphene and Hydra showed reduced performance overhead. In Graphene-op, average performance overhead is reduced by 65.7%. This implies that tracking victims effectively filters unnecessary refresh actions in Graphene. In the case of Hydra-op, the average performance overhead is reduced by 71.7%. This shows that using per-row threshold increases effective $Flip_{th}$, leading to a smaller number of refresh actions.

To understand the source of performance gain, further analyses are done for overhead in designs. Figure 4.3 shows additional refresh done to restore charge in victims (4.3a), and extra DRAM accesses for the RCT counter in Hydra (4.3b). For Graphene, the average number of extra refreshes is reduced by 64.7%, which matches the percentage of reduced performance overhead. The average number of extra refreshes in Hydra is reduced by 98.1%. However, even though extra refreshes were reduced by a significant amount, there were little or no performance benefits in the case of the tracker (4.3b) and showed a larger number of extra DRAM read/write. The main reason for such performance overhead is that the sum of the RCT entry size is doubled due to the row's $Flip_{th}$ metadata using the same bit width as the RCT counter. Extra DRAM read/write occurs when a GCT counter becomes T_G , and all RCT counters are initialized to T_G . Nevertheless, Hydra-op still achieves higher performance because it does very few refreshes.

Chapter 5

Related Work

In this chapter, I discuss currently proposed RH mitigations in detail. Research can be classified into three categories: mitigation using Near-Row-Refresh, mitigations via swapping, and other alternatives

Mitigation using NRR. The majority of prior proposals [1], [8], [4], [7], [3] use counter or probabilistic approach to invoke refresh-based mitigation, where adjacent rows of aggressor row restore leaked charges by row refreshing. Although refreshing victims is a popular choice, the tracker residing in MC is unaware of refreshed victims, leading to security breakage such as Half-Double [12].

Mitigation via Swapping Rows. In order to overcome the security vulnerability of NRR, Saileshwar et al [15] proposed RRS, a novel mechanism of swapping the aggressor row to another random row in the DRAM bank. This breaks the spatial relation between victim and aggressor row, effectively compensating for NRR’s weakness. However, since an attacker can still access swapped rows, RRS must use a threshold of $Flip_{th}/12$ to ensure required level of security. Recently proposed AQUA [16] targets such performance overhead of RRS by setting predefined swap-region inside DRAM. Since all aggressor rows are swapped to swap-region where the attacker can’t access, AQUA does not suffer from a low threshold and achieve higher performance.

Other Alternatives. BlockHammer [17] is an RH mitigation mechanism by throttling aggressor row from accessing DRAM within tREFW after reaching a predefined threshold. It uses a bloom filter to classify rows between malign and benign accesses. Although it is lightweight and incurs small performance overhead, it is susceptible to a denial-of-service attack where an attacker repetitively accesses a shared region in a DRAM such as a shared library or ap-

CHAPTER 5. RELATED WORK

plication. This can lead to a serious slowdown of the entire system. Another promising area of research is manipulating DRAM architecture for stronger security and higher performance, such as CROW [18]. CROW adds additional *copy-row* to each subarray. Copy-rows are used to save aggressor rows at RH attack as well as speed up read/write by faster voltage sensing. Since rows can only be copied to copy-rows at the same subarray, CROW cannot provide a security guarantee for concentrated RH attacks targeting a single subarray.

Chapter 6

Conclusion and Future Work

In this study, I analyzed two state-of-the-art RH mitigation schemes and point out the root cause of their inefficiencies. Utilizing internal DRAM mapping and each row's unique *Flip_{th}*, I designed a more efficient version of each design: Graphene-op and Hydra-op. All of those designs are implemented in Macsim with Ramulator. I evaluated the performance and the number of mitigation actions under benign and malicious memory access. In Graphene, the number of refresh actions for RH mitigation was reduced by 50% in malicious workload and 64.7% in SPEC2017. Due to reduced mitigation, performance overhead was reduced by 65.7% in normal workloads. In the case of Hydra, the number of refreshes was reduced by 91.4% in the attack pattern and 98.1% in normal workloads. Performance overhead was reduced by 71.7% in normal workloads.

Although Hydra-op does not implement victim tracking, it is orthogonal to the current scheme and can be easily extended. Also, Hydra-op can be integrated with a row-swapping mechanism for further performance gain. As discussed in 4.2.2, the major limitation of Hydra-op is extra DRAM access due to additional row metadata in RCT. To remove such performance overhead, we can think of the following scheme: If Hydra-op control logic finds multiple aggressor rows in the same row-group, it can swap aggressors to other rows in a different row-group. This can effectively alleviate contention for a counter in the same row-group.

Reference

- [1] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: An experimental study of dram disturbance errors. In *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, pages 361–372, 2014.
- [2] Jeremie S. Kim, Minesh Patel, A. Giray Yağlıkçı, Hasan Hassan, Roknoddin Azizi, Lois Orosa, and Onur Mutlu. Revisiting rowhammer: An experimental analysis of modern dram devices and mitigation techniques. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture, ISCA '20*, page 638–651. IEEE Press, 2020.
- [3] Yeonhong Park, Woosuk Kwon, Eojin Lee, Tae Jun Ham, Jung Ho Ahn, and Jae W. Lee. Graphene: Strong yet lightweight row hammer protection. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1–13, 2020.
- [4] Moinuddin Qureshi, Aditya Rohan, Gururaj Saileshwar, and Prashant J. Nair. Hydra: Enabling low-overhead mitigation of row-hammer at ultra-low thresholds via hybrid tracking. In *Proceedings of the 49th Annual International Symposium on Computer Architecture, ISCA '22*, page 699–710, New York, NY, USA, 2022. Association for Computing Machinery.
- [5] Hyesoon Kim, Jaekyu Lee, Nagesh B Lakshminarayana, Jaewoong Sim, Jieun Lim, and Tri Pho. Macsim: A cpu-gpu heterogeneous simulation framework user guide. *Georgia Institute of Technology*, 2012.
- [6] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A fast and extensible dram simulator. *IEEE Computer architecture letters*, 15(1):45–49, 2015.

- [7] Eojin Lee, Ingab Kang, Sukhan Lee, G. Edward Suh, and Jung Ho Ahn. Twice: Preventing row-hammering by exploiting time window counters. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, pages 385–396, 2019.
- [8] Seyed Mohammad Seyedzadeh, Alex K. Jones, and Rami Melhem. Counter-based tree structure for row hammering mitigation in dram. *IEEE Computer Architecture Letters*, 16(1):18–21, 2017.
- [9] J. Misra and David Gries. Finding repeated elements. *Science of Computer Programming*, 2(2):143–152, 1982.
- [10] Pietro Frigo, Emanuele Vannacc, Hasan Hassan, Victor van der Veen, Onur Mutlu, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. Trtrespass: Exploiting the many sides of target row refresh. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 747–762, 2020.
- [11] JEDEC Solid State Technology Association. *JEDEC Standard JESD79-4B: Double Data Rate 4 SDRAM (DDR4 SDRAM)*. JEDEC, Virginia, USA, 2017.
- [12] Andreas Kogler, Jonas Juffinger, Salman Qazi, Yoongu Kim, Moritz Lipp, Nicolas Boichat, Eric Shiu, Mattias Nissler, and Daniel Gruss. Half-Double: Hammering from the next row over. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3807–3824, Boston, MA, August 2022. USENIX Association.
- [13] Lois Orosa, Abdullah Giray Yaglikci, Haocong Luo, Ataberk Olgun, Jisung Park, Hasan Hassan, Minesh Patel, Jeremie S. Kim, and Onur Mutlu. A deeper look into rowhammer’s sensitivities: Experimental analysis of real dram chips and implications on future attacks and defenses. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO ’21, page 1182–1197, New York, NY, USA, 2021. Association for Computing Machinery.
- [14] Standard Performance Evaluation Corporation. *SPEC CPU2017 Benchmark Suite*, 2017.
- [15] Gururaj Saileshwar, Bolin Wang, Moinuddin Qureshi, and Prashant J. Nair. Randomized row-swap: Mitigating row hammer by breaking spatial correlation between aggressor and victim rows. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’22, page 1056–1069, New York, NY, USA, 2022. Association for Computing Machinery.

- [16] Anish Saxena, Gururaj Saileshwar, Prashant J. Nair, and Moinuddin Qureshi. Aqua: Scalable rowhammer mitigation by quarantining aggressor rows at runtime. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 108–123, 2022.
- [17] A. Giray Yağlikçi, Minesh Patel, Jeremie S. Kim, Roknoddin Azizi, Ataberk Olgun, Lois Orosa, Hasan Hassan, Jisung Park, Konstantinos Kanellopoulos, Taha Shahroodi, Saugata Ghose, and Onur Mutlu. Blockhammer: Preventing rowhammer at low cost by black-listing rapidly-accessed dram rows. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 345–358, 2021.
- [18] Hasan Hassan, Minesh Patel, Jeremie S. Kim, A. Giray Yaglikci, Nandita Vijaykumar, Nika Mansouri Ghiasi, Saugata Ghose, and Onur Mutlu. Crow: A low-cost substrate for improving dram performance, energy efficiency, and reliability. In *Proceedings of the 46th International Symposium on Computer Architecture, ISCA '19*, page 129–142, New York, NY, USA, 2019. Association for Computing Machinery.

秒錄

DRAM RowHammer 공격은 2014년 [1] 실험적으로 입증된 이후 컴퓨팅 시스템 보안에 중대한 위협으로 작용하였다. RowHammer 공격은 DRAM의 특정 wordline을 반복적으로 활성화시켜 전자기파 간섭으로 인해 주변 다른 row의 축전기에서 전하가 빠져나가도록 유도하는 것으로, 반도체 공정이 점점 더 미세해짐에 따라 더 빈번하게 발생하고 있다 [2]. RowHammer 공격을 막는 다양한 방식이 연구되었지만, DRAM 내부 구조가 공개되지 않았기 때문에 효율 측면에서 한계가 존재하였다. 본 논문에서는 DDR4 내부 구조가 공개되었을 때 제시된 RowHammer 방어전략들 중 가장 최신 방식인 Graphene [3] 과 Hydra [4] 에서 개선할 수 있는 부분들을 제시하였다. RowHammer 공격이 최소 1K의 ACT 명령어에서 축전기의 전하를 충분히 빠져나가게 한다고 가정하고 Maccsim [5] 과 Ramulator [6] (2 channel DDR4) 시뮬레이터를 사용해 2개의 RowHammer 공격 패턴과 SPEC CPU 2017에서 메모리 접근이 가장 많은 3개의 트레이스를 사용해 실험을 진행하였다. 실험 결과 Graphene에서는 추가적인 REF 명령어의 수가 기존에 비해 공격 패턴에서는 50%, SPEC 트레이스에서는 64.7% 감소하였으며 IPC 손실을 65.7% 감소시킬 수 있었다. Hydra의 경우 추가적인 REF 명령어 수가 기존에 비해 공격 패턴에서는 91.4%, SPEC 트레이스에서는 98.1% 감소하였고 IPC 손실을 71.7% 감소시킨 결과를 얻었다.

주요어: DRAM, DDR4, 메모리 신뢰성, RowHammer 공격, Graphene, Hydra